

CPSC 250

Plotting the Credits Profile

Edward Brash

1 Goal

After simulating many games, we want to visualize the results.

In particular, we want to plot the number of credits as a function of round number.

This is a data manipulation task:

Store the data, then plot the data.

2 What Data Do We Need?

To plot credits versus round number, we need two lists:

- x-values: round numbers,
- y-values: credits.

For example:

```
rounds = [0, 1, 2, 3, 4]
credits = [10, 11, 12, 11, 12]
```

These lists must correspond element-by-element.

3 Storing Simulation Results

For one simulation, we might store:

```
result_round = []
credit_round = []
```

At each round:

```
result_round.append(rounds)
credit_round.append(credits)
```

For many simulations, we need lists of lists:

```
round_result = []
credit_result = []
result_number = []
```

Each simulation contributes one list of round numbers and one list of credit values.

4 Matplotlib

Python's most common plotting library is Matplotlib.

The usual import is:

```
import matplotlib.pyplot as plt
```

The `pyplot` part is one submodule of Matplotlib.

By convention, we import it as:

```
plt
```

5 Basic Plot

A basic line plot is:

```
plt.plot(result_round, credit_round)
plt.show()
```

Here:

- `result_round` provides the x-values,
- `credit_round` provides the y-values.

6 Rules of Plotting

A good plot should include:

1. a descriptive title,
2. a labeled x-axis,
3. a labeled y-axis.

For this problem, a good title is:

```
plt.title("Credits vs. Round Number")
```

The axis labels are:

```
plt.xlabel("Round Number")
plt.ylabel("Credits")
```

7 Complete Single-Game Plot

```
plt.plot(result_round, credit_round)

plt.title("Credits vs. Round Number")
plt.xlabel("Round Number")
plt.ylabel("Credits")

plt.show()
```

8 Plotting Multiple Simulations

If we run many simulations, we can plot several credit histories at once.

Suppose:

```
round_result[i]
credit_result[i]
```

store the data for simulation i .

Then:

```
for i in range(num_sims):
    plt.plot(round_result[i], credit_result[i])
```

plots all simulations on the same axes.

9 Interpreting the Plot

Each line represents one simulated game.

A game begins with some initial number of credits.

Then credits move up and down until the game ends at zero credits.

This creates a random-walk-like path.

10 Histogram of Game Lengths

Another useful plot is a histogram of the number of rounds required before the game ends.

If:

```
result_number = [18422, 503, 971, ...]
```

stores the number of rounds for each simulation, then:

```
plt.hist(result_number)
plt.show()
```

creates a histogram.

11 Number of Bins

A histogram groups data into bins.

The number of bins affects the visual interpretation.

For example:

```
plt.hist(result_number, bins=20)
```

uses 20 bins.

Too few bins may hide structure. Too many bins may make the plot noisy.

12 Histogram Labels

As always, we label the plot.

```
plt.title("Distribution of Game Lengths")
plt.xlabel("Number of Rounds")
plt.ylabel("Frequency")
```

13 Complete Histogram Example

```
plt.hist(result_number, bins=20)

plt.title("Distribution of Game Lengths")
plt.xlabel("Number of Rounds")
plt.ylabel("Frequency")

plt.show()
```

14 Philosophy of the Simulation

The notes emphasize that we are not just playing a game.

We are modeling a random walk process.

This connects programming to probability, statistics, and simulation.

The general scientific workflow is:

1. define a model,
2. run the model many times,
3. store the results,
4. visualize the results,
5. interpret the behavior.

15 Key Takeaways

- Plotting requires stored x-values and y-values.
- For the credits simulation, x is round number and y is credits.
- Matplotlib is commonly imported as `plt`.
- Good plots need titles and axis labels.
- Multiple simulations can be plotted on the same axes.
- Histograms show distributions.
- The number of bins controls histogram resolution.